AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

HIGH-PERFORMANCE AND CLOUD COMPUTING

Unit 3: GPU Architecture and CUDA

Why a graphics processor became the engine of scientific computing, how to program one, and how to tell whether your kernel is any good.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

COURSE
23AID304 High-Performance and Cloud Computing

UNIT
3 of 4

SYLLABUS
GPU architecture and evolution, GPGPU and CUDA, CUDA programming, thread execution, matrix computing in CUDA, cuBLAS and cuDNN, case studies

HARDWARE
asaicompute: 4x A100 80GB, 2x RTX 6000 Ada, CUDA 13.0

OFFERING
Odd Semester 2026-27

Why GPUs

A CPU core is designed to make **one** instruction stream finish as fast as possible. Most of its silicon is spent on that goal: branch predictors, out-of-order machinery, large caches, deep pipelines. The arithmetic units are a small fraction of the chip.

A GPU makes the opposite trade. It spends almost everything on arithmetic units and on the register file, and almost nothing on making a single thread fast. It then hides memory latency not with caches but by having so many threads resident that whenever one stalls, another is ready to run.

	CPU (EPYC 9654)	GPU (A100 80GB)
Cores	96 general-purpose	108 SMs × 64 FP32 lanes
Threads in flight	~192	~220 000
Clock	~2.4 GHz	~1.4 GHz
FP64 peak	~4 TFLOP/s	9.7 TFLOP/s (19.5 with tensor cores)
Memory bandwidth	~400 GB/s	~2 000 GB/s
Cache per thread	Megabytes	Bytes
Good at	Branching, latency-sensitive, irregular	Regular, data-parallel, high arithmetic intensity

The bandwidth row is the one that matters most: an A100 delivers roughly five times the memory bandwidth of a large server CPU, and that is what makes it worth the trouble of moving your data across a PCIe bus to get to it.

DEFINITION · GPGPU

General-Purpose computing on Graphics Processing Units: using a GPU for computation unrelated to rendering. Before CUDA (2007), people did this by disguising their problem as a graphics operation, encoding data as textures and results as pixel colours. CUDA replaced that with a real programming model, and the field changed within about three years.

BY THE END OF THIS UNIT YOU WILL BE ABLE TO

- Explain the SIMT execution model and what a warp is. - Write, launch and debug a CUDA kernel, with correct indexing. - Choose block and grid dimensions, and reason about occupancy. - Explain memory coalescing and write kernels that achieve it. - Use shared memory to tile a matrix multiplication and say why it helps. - Perform a parallel reduction on a GPU without a race. - Overlap transfer and compute using streams. - Call cuBLAS and cuDNN, and explain when to prefer them over your own kernel. - Profile a kernel and decide whether it is memory or compute bound.

The execution model

Threads, blocks, grids

A CUDA kernel is launched over a **grid** of **thread blocks**, each containing many **threads**. This hierarchy is not decoration; each level maps onto hardware.

```
grid  ──────────────────────────────────────────────
  ├── block (0,0)   ├── block (1,0)   ├── block (2,0)
  │   ├ thread      │                 │
  │   ├ thread      │   up to 1024 threads per block
  │   └ ...         │   threads in a block can co-operate:
  └── ...           │     shared memory + __syncthreads()
                    │   blocks CANNOT co-operate
```

LEVEL	MAPS TO	CAN CO-OPERATE?
Thread	One lane of an SM	With others in its block
Warp (32 threads)	Scheduled together, one instruction	Implicitly, in lockstep
Block	One **streaming multiprocessor (SM)**	Via shared memory and `__syncthreads()`
Grid	The whole GPU	No. Blocks must be independent

DEFINITION · WHY BLOCKS MUST BE INDEPENDENT

The runtime may schedule blocks in any order, concurrently or one after another, on any SM. A program that assumes block 5 runs after block 4 works on one GPU and fails on another. This independence is what lets the same binary scale from a laptop GPU with 20 SMs to an A100 with 108.

SIMT and warp divergence

Threads execute in groups of 32 called **warps**. All 32 share one instruction pointer. This is SIMD hardware wearing a threaded interface, which NVIDIA calls **SIMT**.

The consequence appears the moment you write an `if`:

```
if (threadIdx.x % 2 == 0)
    heavy_a();          /* odd  threads masked off, idle */
else
    heavy_b();          /* even threads masked off, idle */
```

The warp executes **both** branches in sequence, masking the inactive threads. Cost is the sum, not the maximum. This is **warp divergence**.

PITFALL · DIVERGENCE IS PER WARP, NOT PER BLOCK

Branching on `blockIdx.x` costs nothing: every thread in the warp agrees. Branching on `threadIdx.x / 32` also costs nothing: whole warps take the same path. Branching on `threadIdx.x % 2` costs double. When you must branch, try to arrange the condition so that it is uniform across each group of 32.

WORKED EXAMPLE · RESTRUCTURING TO REMOVE DIVERGENCE

```c / *diverges: adjacent threads take different paths* / if (data[tid] > threshold) out[tid] = expensive(data[tid]);

/ *better: sort or partition the data first so that whole warps agree, or compute both and select, if both are cheap* / float v = expensive(data[tid]); out[tid] = (data[tid] > threshold) ? v : out[tid]; ```
The second form does more arithmetic and less branching, which on a GPU is usually the right trade.

## Your first kernel

```c
#include <cuda_runtime.h>
#include <stdio.h>

__global__ void vec_add(const float *a, const float *b, float *c, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;   /* the standard idiom */
    if (i < n) c[i] = a[i] + b[i];                   /* the guard is essential */
}

int main(void)
{
    int n = 1 << 20;
    size_t bytes = n * sizeof(float);

    float *ha = malloc(bytes), *hb = malloc(bytes), *hc = malloc(bytes);
    for (int i = 0; i < n; i++) { ha[i] = i; hb[i] = 2.0f * i; }

    float *da, *db, *dc;
    cudaMalloc(&da, bytes); cudaMalloc(&db, bytes); cudaMalloc(&dc, bytes);
    cudaMemcpy(da, ha, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(db, hb, bytes, cudaMemcpyHostToDevice);

    int threads = 256;
    int blocks  = (n + threads - 1) / threads;       /* ceiling division */
    vec_add<<<blocks, threads>>>(da, db, dc, n);

    cudaMemcpy(hc, dc, bytes, cudaMemcpyDeviceToHost);
    cudaFree(da); cudaFree(db); cudaFree(dc);
    return 0;
}
```

Three details that catch everyone:
```

PITFALL · THE THREE BEGINNER ERRORS

1. Missing bounds guard. `blocks * threads` is usually larger than `n` because of the ceiling division, so without `if (i < n)` the extra threads write past the end of the array. This corrupts memory silently.

2. Assuming the kernel finished. A kernel launch is **asynchronous**: it returns immediately. `cudaMemcpy` happens to synchronise, which hides the issue, but timing code around a launch without `cudaDeviceSynchronize()` measures the launch, not the work.

3. Not checking errors. Kernel launch failures are reported *asynchronously*, so the error surfaces at some later, unrelated call. Always wrap calls:

```c #define CUDA_CHECK(call) do { \ cudaError_t e = (call); \ if (e != cudaSuccess) { \ fprintf(stderr, "%s:%d: %s\n", **FILE**, **LINE**, \ cudaGetErrorString(e)); \ exit(1); \ } } while (0)

vec_add<<>>(da, db, dc, n); CUDA_CHECK(cudaGetLastError()); / *catches launch configuration errors* / CUDA_CHECK(cudaDeviceSynchronize()); / *catches errors during execution* / ```
```

The GPU memory hierarchy

Unit 0's hierarchy again, with different names and much sharper trade-offs.

SPACE	SCOPE	LATENCY	SIZE	NOTES
Registers	One thread	~1 cycle	255 per thread max	Fastest; too many cuts occupancy
Shared memory	One block	~20 cycles	up to 164 KB per SM	Programmer-managed cache. The key resource
L1 / texture	One SM	~30 cycles	shares the shared-memory pool	Automatic
L2	Whole GPU	~200 cycles	40 MB on A100	Automatic
Global	Whole GPU, persists	~400 cycles	80 GB on A100	Where your data lives
Constant	Read-only, broadcast	~cached	64 KB	Good for coefficients all threads read
Host memory	CPU	~microseconds over PCIe	System RAM	The slowest path by far

PITFALL · PCIE IS THE BOTTLENECK NOBODY PLANS FOR

PCIe 4.0 x16 gives about 25 GB/s. The A100's own memory delivers 2 000 GB/s, eighty times more. Copying data to the GPU, running one cheap kernel, and copying it back is *slower* than doing it on the CPU. GPUs pay off when data stays resident and many kernels run on it.

Rule of thumb: if your kernel's arithmetic intensity is below about 10 FLOP per byte transferred over PCIe, question whether the GPU is the right tool.

Coalescing

This is the single most important performance rule on a GPU, and it is Unit 0's cache-line argument again.

When the 32 threads of a warp access global memory, the hardware combines their requests into as few memory transactions as it can. If the threads read 32 *consecutive* 4-byte words, that is one 128-byte transaction: perfectly **coalesced**. If they read 32 scattered words, that can be 32 separate transactions, and you use 4 bytes of every 32-byte sector fetched.

```c
/* COALESCED: consecutive threads touch consecutive addresses */
int i = blockIdx.x * blockDim.x + threadIdx.x;
out[i] = in[i];

/* NOT COALESCED: consecutive threads are 'stride' apart */
out[i * stride] = in[i * stride];        /* up to 32x the memory traffic */
```

WORKED EXAMPLE · MATRIX TRANSPOSE, THE CLASSIC COALESCING EXERCISE

`c /* naive: reads coalesced, writes strided */ __global__ void transpose_naive(float *out, const float *in, int n) { int x = blockIdx.x * TILE + threadIdx.x; int y = blockIdx.y * TILE + threadIdx.y; out[x * n + y] = in[y * n + x]; /* the write strides by n */ }` One of the two accesses must be strided, because transposition inverts the index order. The fix is to route the data through shared memory so that *both* global accesses are coalesced and the awkward transposition happens on chip:

```c **global** void transpose_tiled(float *out, const float* in, int n) { **shared** float tile[TILE][TILE + 1]; /* +1 avoids bank conflicts */

```c
int x = blockIdx.x * TILE + threadIdx.x;
int y = blockIdx.y * TILE + threadIdx.y;
tile[threadIdx.y][threadIdx.x] = in[y * n + x];   /* coalesced read  */

__syncthreads();

x = blockIdx.y * TILE + threadIdx.x;               /* swapped blocks  */
y = blockIdx.x * TILE + threadIdx.y;
out[y * n + x] = tile[threadIdx.x][threadIdx.y];   /* coalesced write */
```

} `` Typical improvement: **five to ten times**. The `TILE + 1` padding is not a typo; it is explained next.

DEFINITION · SHARED MEMORY BANK CONFLICTS

Shared memory is divided into 32 **banks**, four bytes wide, interleaved. Thirty-two threads accessing thirty-two different banks proceed in parallel. If two threads in a warp hit *different addresses in the same bank*, the accesses serialise.

A `__shared__ float tile[32][32]` has `tile[0][k]` and `tile[1][k]` exactly 32 floats apart, so a whole column lands in one bank: a 32-way conflict. Declaring `tile[32][33]` shifts each row by one, spreading a column across all 32 banks. One character, often a two-fold speedup.
```

Tiled matrix multiplication

The set piece of GPU programming, and the direct descendant of cache blocking in Unit 2.

Naive version

```
__global__ void matmul_naive(float *C, const float *A, const float *B, int n)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < n && col < n) {
        float s = 0.0f;
        for (int k = 0; k < n; k++)
            s += A[row * n + k] * B[k * n + col];
        C[row * n + col] = s;
    }
}
```

Every thread reads a full row of A and a full column of B from global memory: $2n$ loads for n multiply-adds, so arithmetic intensity is about 0.25 FLOP/byte. Unit 1's roofline says immediately that this is hopeless.

Tiled version

Each block cooperatively loads a TILE × TILE tile of A and of B into shared memory, then every thread in the block uses those tiles many times.

```cuda
#define TILE 32

__global__ void matmul_tiled(float *C, const float *A, const float *B, int n)
{
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int tx = threadIdx.x, ty = threadIdx.y;
    int row = blockIdx.y * TILE + ty;
    int col = blockIdx.x * TILE + tx;

    float acc = 0.0f;

    for (int t = 0; t < (n + TILE - 1) / TILE; t++) {
        /* cooperative, coalesced load of one tile from each matrix */
        int aCol = t * TILE + tx;
        int bRow = t * TILE + ty;
        As[ty][tx] = (row < n && aCol < n) ? A[row * n + aCol] : 0.0f;
        Bs[ty][tx] = (bRow < n && col < n) ? B[bRow * n + col] : 0.0f;

        __syncthreads();                /* tiles are now filled */

        for (int k = 0; k < TILE; k++)
            acc += As[ty][k] * Bs[k][tx];

        __syncthreads();                /* before overwriting the tiles */
    }

    if (row < n && col < n) C[row * n + col] = acc;
}
```

Each element loaded into shared memory is used TILE times instead of once, so global traffic falls by a factor of TILE and arithmetic intensity rises from about 0.25 to about TILE/4 FLOP per byte. With TILE = 32 that is a factor of 32, which moves the kernel from far left of the ridge point to comfortably right of it.

PITFALL · THE TWO __SYNCTHREADS() ARE BOTH NECESSARY

The first ensures every thread has finished *writing* its tile element before anyone *reads* the tile. The second ensures every thread has finished *reading* the tile before the next iteration *overwrites* it. Removing either gives a race that produces wrong answers intermittently, and more often on larger matrices.

Also: `__syncthreads()` must be reached by **every** thread in the block. Putting it inside a divergent `if` hangs the block.

Here is what that actually buys, measured on one RTX 6000 Ada of `asaicompute`, 4096 × 4096 single precision, best of three runs:

VERSION	TIME	GFLOP/S	SPEEDUP OVER NAIVE
Naive	24.42 ms	5 628	1.00x
Tiled, TILE = 32	19.44 ms	7 071	**1.26x**
cuBLAS `SGEMM`	2.17 ms	63 302	**11.25x**

COMMON MISCONCEPTION · TILING GIVES A TEN-FOLD SPEEDUP

Every textbook says so, and on the hardware those textbooks were written for it was true. Measured on current hardware it is **1.26x**, and you should know why the number moved.

Two things changed. First, this GPU has a very large L2 cache, so most of the re-reads the naive kernel performs are served from L2 rather than from DRAM. Tiling is saving traffic that was no longer expensive. Second, the naive kernel here is already **coalesced**: within a warp all 32 threads share the same `row`, so `A[row*k+t]` is one broadcast address, and `col` is consecutive, so `B[t*n+col]` is a perfectly coalesced read. The naive kernel is not the disaster its reputation suggests.

Tiling is still the right thing to write, and the technique generalises to problems where L2 cannot rescue you. But quoting a speedup from a ten-year-old book, without measuring, is how people end up optimising the wrong thing. **Measure on the machine you are using.**

The cuBLAS row is the real lesson. A tuned library is roughly **nine times faster than your best hand-written tiled kernel**, because it also does register blocking, software pipelining, and, on this hardware, uses TF32 tensor cores for single precision by default. That last point matters when comparing: cuBLAS is not doing the same arithmetic your kernel is.

Parallel reduction on a GPU

Summing an array is trivial on a CPU and instructive on a GPU, because there is no global synchronisation between blocks.

```
__global__ void reduce_sum(const float *in, float *out, int n)
{
    extern __shared__ float sdata[];
    int tid = threadIdx.x;
    int i   = blockIdx.x * blockDim.x * 2 + threadIdx.x;

    /* each thread loads and adds two elements: halves the block count */
    float v = (i < n) ? in[i] : 0.0f;
    if (i + blockDim.x < n) v += in[i + blockDim.x];
    sdata[tid] = v;
    __syncthreads();

    /* tree reduction in shared memory, no divergence, no bank conflicts */
    for (unsigned s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) sdata[tid] += sdata[tid + s];
        __syncthreads();
    }

    if (tid == 0) out[blockIdx.x] = sdata[0];   /* one partial per block */
}
```

Each block produces one partial sum; a second, tiny kernel launch reduces those.

BEYOND THE SYLLABUS · WARP-LEVEL PRIMITIVES

Below 32 elements the threads are all in one warp and already implicitly synchronised, so `__syncthreads()` is wasted. Modern CUDA exposes shuffle instructions that exchange registers directly between lanes:

```
c for (int offset = 16; offset > 0; offset >>= 1) v += __shfl_down_sync(0xffffffff, v, offset);
/* lane 0 of each warp now holds the warp's total */
```

This skips shared memory entirely for the final steps and is measurably faster. Note `__shfl_down` without `_sync` is removed in current CUDA: the mask argument is mandatory because Volta and later allow threads within a warp to diverge independently.

Streams and overlap

By default everything runs in stream 0 and serialises: copy in, compute, copy out. With multiple streams you can overlap the transfer of one chunk with the computation of another, hiding most of the PCIe cost.

```
cudaStream_t s[NSTREAMS];
for (int k = 0; k < NSTREAMS; k++) cudaStreamCreate(&s[k]);

int chunk = n / NSTREAMS;
for (int k = 0; k < NSTREAMS; k++) {
    int off = k * chunk;
    cudaMemcpyAsync(d + off, h + off, chunk * sizeof(float),
                    cudaMemcpyHostToDevice, s[k]);
    kernel<<<blocks, threads, 0, s[k]>>>(d + off, chunk);
    cudaMemcpyAsync(h + off, d + off, chunk * sizeof(float),
                    cudaMemcpyDeviceToHost, s[k]);
}
cudaDeviceSynchronize();
```

PITFALL · ASYNC COPIES NEED PINNED HOST MEMORY

`cudaMemcpyAsync` from ordinary `malloc` memory silently behaves synchronously, because the driver must first stage it through a pinned buffer. Allocate host memory with `cudaMallocHost` (page-locked) for genuine overlap. Do not pin everything: pinned memory cannot be swapped, and over-pinning degrades the whole system.

Occupancy

DEFINITION · OCCUPANCY

The ratio of resident warps per SM to the maximum the SM supports. It measures how much latency-hiding capacity you have, because when one warp stalls on memory the SM switches to another.

Occupancy is limited by whichever resource runs out first:

- **Registers per thread.** 255 max; using many means fewer resident threads.
- **Shared memory per block.** A block using 48 KB on an SM with 164 KB limits you to three blocks.
- **Threads per block.** 1024 max; 128 to 512 is the usual sweet spot.
- **Blocks per SM.** Hardware limit, typically 16 to 32.

```
nvcc -O3 -arch=sm_80 --ptxas-options=-v kernel.cu -o kernel
# ptxas info: Used 38 registers, 8192 bytes smem, 380 bytes cmem[0]
```

COMMON MISCONCEPTION · HIGHER OCCUPANCY ALWAYS MEANS BETTER PERFORMANCE

It does not. Occupancy is a means, not an end: it exists to hide memory latency. A kernel that uses many registers per thread to keep values on chip may run at 25 percent occupancy and beat a 100 percent occupancy version that reloads everything from memory. Volkov's well-known result showed exactly this for matrix multiplication. Aim for *enough* occupancy to hide latency, then optimise for memory traffic and instruction mix.

Libraries: cuBLAS and cuDNN

DEFINITION · CUBLAS

NVIDIA's implementation of BLAS, the Basic Linear Algebra Subprograms: Level 1 vector operations, Level 2 matrix-vector, Level 3 matrix-matrix. `cublasSgemm` and `cublasDgemm` are the workhorses.

```
cublasHandle_t h;
cublasCreate(&h);
const float alpha = 1.0f, beta = 0.0f;
/* C = alpha * A*B + beta * C  */
cublasSgemm(h, CUBLAS_OP_N, CUBLAS_OP_N,
            n, n, n, &alpha, dA, n, dB, n, &beta, dC, n);
cublasDestroy(h);
```

PITFALL · CUBLAS IS COLUMN-MAJOR

cuBLAS inherits Fortran's column-major convention while C is row-major. The common trick is to exploit the identity that computing $B^T \cdot A^T$ in column-major order gives you $A \cdot B$ in row-major order, so you pass the matrices in swapped order and read the result directly. Getting this wrong produces a transposed answer that looks plausible.

DEFINITION · CUDNN

The deep-learning primitive library: convolutions, pooling, normalisation, activations, recurrent cells. It selects among several convolution algorithms (implicit GEMM, FFT, Winograd) based on the tensor shapes. Every major framework calls into it; when you run PyTorch on a GPU, cuDNN is doing the work.

BEYOND THE SYLLABUS · TENSOR CORES AND THE TILE PROGRAMMING MODEL

From Volta onward, NVIDIA GPUs contain **tensor cores**: units that compute a small matrix multiply-accumulate in one instruction, at several times the throughput of the general FP32 units, usually with reduced-precision inputs and FP32 accumulation. They are the reason an A100 quotes 19.5 TFLOP/s FP64 with tensor cores against 9.7 without.

Programming them directly through the WMMA API is awkward. CUDA 13.0 added a second, complementary programming model alongside the classic thread-parallel one: **tile-based programming**, extended in CUDA 13.1 as **CUDA Tile**. Instead of writing what each thread does, you describe operations on *tiles* of data and the compiler maps them onto tensor cores and manages tile memory. The stated aim is that code written this way remains valid on future tensor-core generations without a rewrite. There is also `cuTile` for Python. Initial support is limited to Blackwell-class GPUs (compute capability 10.x and 12.x), so it will not run on the A100s in `asaicompute`, but it is where the model is heading and you should know it exists.

PITFALL · WRITE THE LIBRARY CALL FIRST

Before writing a custom kernel, check whether cuBLAS, cuDNN, cuFFT, cuSPARSE, Thrust or CUB already does it. They will be faster than your first ten attempts. Write custom kernels for the parts that have no library equivalent, which in practice is the problem-specific glue between library calls.

Profiling

Guessing is not a method. NVIDIA's tools tell you exactly where the time goes.

```
nsys profile -o report ./myapp          # timeline: kernels, copies, gaps
nsys stats report.nsys-rep

ncu --set full -o kernel_report ./myapp # per-kernel counters
ncu --metrics sm__throughput.avg.pct_of_peak_sustained_elapsed,\
dram__throughput.avg.pct_of_peak_sustained_elapsed ./myapp
```

Read them in this order:

1. **Nsight Systems first.** Is the GPU even busy? A timeline full of gaps means the problem is host-side: too-small kernels, missing overlap, synchronous copies. No kernel optimisation will help.
2. **Nsight Compute second**, once a specific kernel is known to dominate. It produces a roofline chart directly, so Unit 1's model appears as a measured plot rather than a hand calculation.

WHAT THE PROFILER SAYS	DIAGNOSIS	ACTION
DRAM throughput near peak, SM low	Memory bound at the limit	Raise intensity: tile, fuse, lower precision
Both low	Latency bound	Raise occupancy, more work per thread, check dependency chains
SM near peak	Compute bound	You are close to done
High "uncoalesced global accesses"	Access pattern	Restructure indexing; stage through shared memory
High warp stall on `__syncthreads`	Imbalance in the block	Rebalance work across threads

Case studies

WORKED EXAMPLE · CASE STUDY 1: MOLECULAR DYNAMICS

Each timestep computes forces between particles. With a cut-off radius the problem becomes a neighbour-list traversal: irregular, but with high arithmetic intensity because each pair interaction is several dozen flops on a few loaded coordinates.

GPU implementations (GROMACS, LAMMPS, OpenMM) keep all coordinates resident in GPU memory across thousands of steps, so PCIe cost is paid once. Speedups of 20 to 100 times over a CPU socket are routine. The engineering difficulty is the neighbour list: rebuilt periodically, sorted spatially so that threads in a warp handle nearby particles and therefore read nearby memory. Coalescing again.

WORKED EXAMPLE · CASE STUDY 2: DEEP LEARNING TRAINING

A transformer's forward and backward passes are dominated by large dense matrix multiplications, which is the single best case for a GPU: very high arithmetic intensity, perfectly regular, and exactly what tensor cores exist for.

The interesting engineering is no longer the kernel, which cuBLAS and cuDNN handle, but everything around it: keeping the input pipeline fed so the GPU never idles, using mixed precision to halve memory traffic, and, once the model exceeds one GPU's memory, splitting it across devices. That last problem is Unit 4.

WORKED EXAMPLE · CASE STUDY 3: WHEN THE GPU LOSES

A sparse graph traversal with unpredictable branching, low arithmetic intensity, and random memory access is close to the worst case for a GPU: every warp diverges, every access is uncoalesced, and there is little arithmetic to hide the latency. A well-tuned CPU implementation with a large cache can win outright.

Knowing when *not* to use an accelerator is as valuable as knowing how.

Self-assessment

1. Give two architectural differences between a CPU core and a GPU SM, and explain the design goal behind each. 2. What is a warp? Why does `if (threadIdx.x % 2)` cost more than `if (blockIdx.x % 2)`? 3. Write the standard global thread index expression, and explain why the bounds guard is necessary. 4. Why can two thread blocks not synchronise with each other, and what does that buy the hardware? 5. A kernel launch is asynchronous. Give one bug and one measurement error this causes. 6. Define coalescing. What is the worst-case traffic multiplier for a warp reading 32 scattered floats? 7. Why must a shared-memory transpose tile be declared `[TILE][TILE+1]`? 8. In the tiled matmul, explain what each of the two `__syncthreads()` prevents. 9. Tiling raises arithmetic intensity from 0.25 to about TILE/4. Explain the mechanism, and connect it to the roofline model. 10. Why does the GPU reduction kernel produce one partial sum per block rather than a single total? 11. What must be true of host memory for `cudaMemcpyAsync` to overlap genuinely? 12. Define occupancy and list three things that limit it. 13. Give an example where lower occupancy gives higher performance. 14. PCIe delivers 25 GB/s; A100 memory delivers 2 000 GB/s. What does this imply about how you should structure a GPU application? 15. cuBLAS is column-major and C is row-major. What is the standard trick? 16. Your profile shows DRAM throughput at 92 percent of peak and SM throughput at 9 percent. What kind of kernel is this, and what are your options? 17. Your Nsight Systems timeline shows the GPU idle 70 percent of the time. Where do you look, and why is optimising the kernel the wrong move? 18. Give one problem class where a CPU beats a GPU, and explain why. 19. What are tensor cores and why did the tile-based programming model appear? 20. Why does `__shfl_down_sync` require an explicit mask?

---

1. A CPU spends most of its area on latency-reduction machinery: large caches, branch prediction, out-of-order execution, so that one instruction stream finishes quickly. A GPU spends most of its area on arithmetic units and registers, and hides latency by keeping thousands of threads resident so another is always ready. The goals are minimum latency versus maximum throughput. **2.** A warp is 32 threads that share one instruction pointer and execute in lockstep. `threadIdx.x % 2` splits threads *within* a warp, so the hardware runs both branches and masks half the lanes each time, costing the sum. `blockIdx.x % 2` is uniform across every thread of a warp, so no divergence occurs. **3.** `int i = blockIdx.x * blockDim.x + threadIdx.x;`. Grid size is computed by ceiling division, so the last block usually contains threads with `i >= n`. Without `if (i < n)` those threads write outside the allocation, corrupting memory silently. **4.** Blocks may be scheduled in any order, concurrently or sequentially, on any SM, and there is no guarantee that two given blocks are resident at the same time, so a wait would deadlock. In exchange, the same binary scales across GPUs with different SM counts without change. **5.** Bug: reading results on the host before the kernel finished, without `cudaDeviceSynchronize()`. Measurement error: timing around a launch measures only the launch overhead, reporting an impossibly fast kernel. **6.** Coalescing is the hardware combining the memory requests of a warp into the fewest possible transactions; consecutive threads accessing consecutive addresses merge into one. Worst case, 32 scattered 4-byte reads become 32 separate 32-byte sector fetches, so **32 times** the useful traffic. **7.** With `[TILE][TILE]` and TILE = 32, elements of a column are exactly 32 floats apart, which places the whole column in one of the 32 shared-memory banks and serialises a warp's column access 32 ways. The extra column shifts each row by one element so a column spreads across all banks. **8.** The first ensures all threads have written their tile elements before any thread reads the tile. The second ensures all threads have finished reading the tile before the next loop iteration overwrites it. Removing either creates a race with intermittently wrong results. **9.** Without tiling each element loaded from global

memory is used once. With tiling each element is loaded into shared memory once and used TILE times by the threads of the block, so *Q* falls by a factor of TILE while *W* is unchanged, and *I* = *W*/*Q* rises proportionally. On the roofline this moves the kernel from the sloped bandwidth-limited region to the flat compute-limited region. **10.** Because there is no synchronisation between blocks, a block cannot wait for other blocks' contributions. Each block reduces what it owns and writes one partial; a second kernel launch, which provides an implicit global barrier, reduces the partials. **11.** It must be page-locked (pinned), allocated with `cudaMallocHost` or `cudaHostAlloc`. From pageable memory the driver stages through an internal pinned buffer and the copy behaves synchronously. **12.** Occupancy is resident warps per SM divided by the maximum supported. Limits: registers per thread, shared memory per block, threads per block, and the hardware cap on resident blocks per SM. **13.** A matrix-multiply kernel where each thread keeps several output values in registers uses many registers, lowering occupancy, but performs far more arithmetic per byte loaded. It beats a high-occupancy version that re-reads operands from memory, because the bottleneck was bandwidth, not latency. **14.** Data must stay resident on the GPU across many kernel launches. Transferring per kernel wastes eighty times the bandwidth; a GPU pays off only when the transfer cost is amortised over a lot of on-device work. **15.** Pass the matrices in swapped order: computing what cuBLAS sees as $B^T \cdot A^T$ in column-major layout produces $A \cdot B$ in row-major layout, so the result can be read directly without an explicit transpose. **16.** It is memory bound and already running at the bandwidth limit. Options: raise arithmetic intensity by tiling or by fusing this kernel with a neighbour so data is loaded once and used twice, or reduce bytes moved by using a smaller data type. Do not optimise the arithmetic. **17.** Look at the host side: kernels too small to fill the GPU, synchronous copies, missing streams, or a data pipeline that cannot feed the device. With the GPU idle 70 percent of the time, even making every kernel infinitely fast would recover only 30 percent of the runtime. **18.** Sparse graph traversal, or any workload with unpredictable branching, low arithmetic intensity and random access. Every warp diverges, no access coalesces, and there is too little arithmetic to hide the latency, while a CPU's large caches and strong branch prediction suit it well. **19.** Tensor cores are hardware units performing a small matrix multiply-accumulate in one instruction at much higher throughput than the general units. The tile-based model appeared because programming them directly is intricate and architecture specific; describing computation on tiles lets the compiler target current and future tensor-core generations without the source being rewritten. **20.** Since Volta, threads within a warp can diverge and reconverge independently, so the compiler can no longer assume all 32 lanes are active at a shuffle. The mask states explicitly which lanes take part, making the operation well defined under independent thread scheduling.

Where to go next

Unit 4 moves outward: from one GPU to many, and from one machine to a datacentre. The tiling and coalescing arguments become collective communication and data parallelism, and the cloud provides the machines. The bandwidth-versus-compute reasoning you have used in every unit so far is exactly what decides how a large model is split across GPUs.